

摘要

占主导地位的序列转换模型是基于复杂的循环或卷积神经网络，包括编码器和解码器。性能最好的模型也通过注意机制连接编码器和解码器。我们提出了一种新的简单网络架构——Transformer，它完全基于注意力机制，完全摒弃了递归和卷积。在两个机器翻译任务上的实验表明，这些模型在质量上更优，同时具有更强的并行性，需要的训练时间显著减少。我们的模型在 WMT 2014 英德翻译任务中获得 28.4 个 BLEU，比现有的最佳效果(包括集合)提高了 2 个 BLEU。在 WMT 2014 英法翻译任务中，我们的模型在 8 个 GPU 上经过 3.5 天的训练后，建立了一个新的单模最先进的 BLEU，得分为 41.8，这只是文献中最好的模型训练的一小部分。我们通过将 Transformer 成功地应用于具有大量和有限训练数据的英语分组解析，证明它可以很好地推广到其他任务。

1. 引言

循环神经网络，特别是长短期记忆[13]和门控循环[7]神经网络，作为序列建模和转导问题(如语言建模和机器翻译)的最先进的方法已经被牢牢地建立起来[35, 2, 5]。此后，大量的努力继续推动循环语言模型和编码器-解码器架构的边界[38, 24, 15]。

*平等的贡献。列举顺序是随机的。Jakob 建议用自注意取代 RNNs，并开始尝试对这一想法进行评估。Ashish 和 Illia 一起设计并实施了第一次 Transformer 模型，并参与了这项工作的方方面面。Noam 提出了尺度点积注意(scaled dot-product attention)、多头注意和无参数位置表示，并成为几乎涉及每一个细节的人。Niki 在我们的原始代码库和 tensor2tensor 中设计、实现、调试和评估了无数的模型变体。Llion 还试验了新的模型变体、负责我们的初始代码库以及高效的推理和可视化。Lukasz 和 Aidan 花费了无数的时间来设计和实现 tensor2tensor 的各个部分，替换了我们之前的代码库，极大地改善了结果并大大加快了我们的研究。

在 Google Brain 期间完成的工作。

在 Google Research 期间完成的工作

第 31 届神经信息处理系统会议(NIPS 2017)，长滩，加州，美国。

循环模型通常根据输入和输出序列的符号位置进行计算。在计算时间内将位置与步长对齐，它们生成一系列的隐藏状态序列 h_t ，就像一个函数：前一个隐藏状态 h_{t-1} 作为位置 t 的输入。这种固有的顺序性阻止了训练示例中的并行化，这在较长的序列长度时变得至关重要，因为内存约束限制了跨示例的批处理。最近的工作通过分解技巧[21]和条件计算[32]在计算效率上取得了显著的提高，同时也提高了后者的模型性能。然而，顺序计算的基本约束仍然存在。

在各种任务中，注意力机制已经成为序列建模和转导模型中令人信服且不可分割的组成部分，它允许在不考虑输入或输出序列距离的情况下对依赖性进行建模[2, 19]。然而，在除了少数情况下[27]，这种注意机制是与循环网络结合使用的。

在本工作中，我们提出了 Transformer，它是一种避开递归得模型架构，而且是完全依赖注意力机制来描述输入和输出之间的全局依赖关系。Transformer 显然支持更多的并行化，在 8 个 P100 GPUs 上经过 12 个小时的训练后，可以在翻译质量上达到一个新的水平。

2. 背景

减少顺序计算的目标也构成了扩展神经 GPU[16]、ByteNet[18]和 ConvS2S[9]的基础，它们都使用卷积神经网络作为基本的构建块，并且是并行计算所有输入和输出位置的隐式表示。在这些模型中，将来自两个任意输入输出位置的信号关联起来所需的操作次数，按照位置之间的距离增长，对 ConvS2S 来说是线性增长，对 ByteNet 来说是对数增长。这使得学习遥远位置之间的依赖关系变得更加困难[12]。在 Transformer 中，这被缩减为一个固定次数的操作，尽管这是以降低有效分辨率为代价的，为均衡注意力权重位置，我们在 3.2 节中描述的多头注意力抵消了这一影响。

自注意力，有时也被称为内部注意，是一种将单个序列的不同位置联系起来以计算序列的表示形式的注意力机制。自注意已成功应用于各种任务，包括阅读理解、抽象摘要、文本概括和学习任务独立的句子表征[4, 27, 28, 22]。

端到端记忆网络是基于一种循环注意机制而不是顺序排列的循环注意机制，并且在简单语言问答和语言建模任务中表现良好[34]。

然而，就我们所知，Transformer 是第一个完全依赖于自注意而不使用序列对齐的 rnn 或卷积来计算其输入和输出表示的转导模型。在下一节中，我们将描述 Transformer，激发自注意，并讨论它相对于[17, 18]和[9]等模型的优势。

3. 模型结构

最具竞争力的神经序列转导模型都有一个编码器-解码器结构[5, 2, 35]。在这里，编码器将一个符号表示的输入序列 $(x_1, \dots, x_n)$ 映射为连续表示的序列 $z = (z_1, \dots, z_n)$ 。对于给定的 z ，解码器一次生成一个符号的输出序列 $(y_1, \dots, y_m)$ 作为某个时间点的元素。模型在每一个步骤中都是自回归的[10]，在生成下一个序列时，使用之前生成的符号作为额外输入。

Transformer 遵循使用堆叠的自注意力层和逐点这种整体架构，编码器和解码器使用全连接层，分别如图 1 的左、右两部分所示

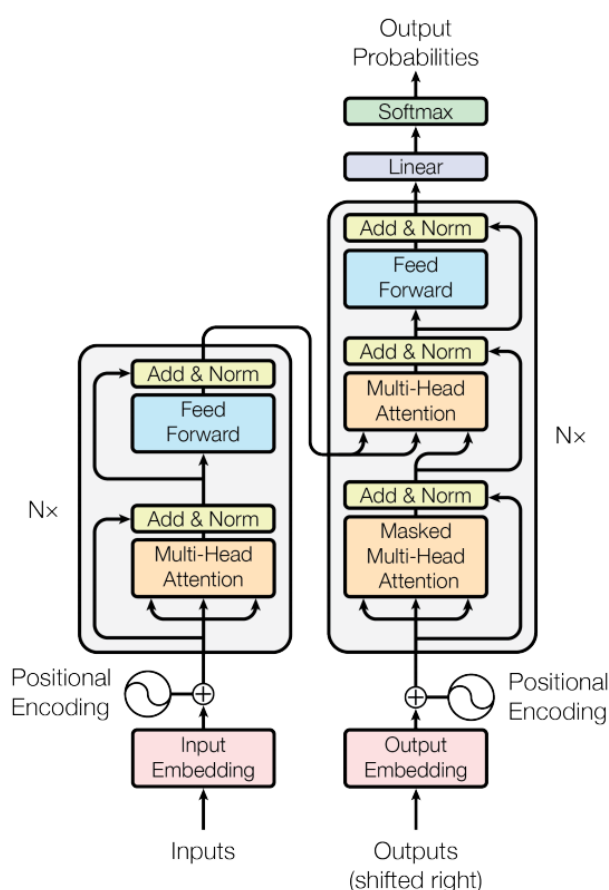


图 1:Transformer 模型架构。

3.1 编码器和解码器堆叠

编码器：编码器由 $N = 6$ 个相同层的堆叠组成。每个层有两个子层。第一个子层是多头自注意力机制，第二个子层是简单的位置全连接前馈网络。我们在这两个子层的每个周围都使用了一个残差连接[11]，然后是归一化层[1]。也就是说，每个子层的输出都是 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中 $\text{Sublayer}(x)$ 是子层本身实现的函数。为了刺激这些残差连接，模型中的所有子层以及嵌入层都产生了 dimension $d_{\text{model}} = 512$ 的输出。

解码器：解码器也由 $N = 6$ 个相同层的堆叠组成。除了每个编码器层中的两个子层外，解码器还插入了第三个子层，该子层对编码器堆的输出进行多头关注。与编码器类似，我们在每个子层周围使用残差连接，然后是归一化层。我们还修改了解码器堆中的自注意力子层，以防止某个位置对后面位置的影响。这种掩码，结合了输出偏移一个位置的事实，确保了位置 i 的预测只能依赖于位置小于 i 的已知输出。

3.2 注意力

注意函数可以描述为将查询和一组键值对映射到输出，其中查询、键、值和输出都是向量。输出是作为一个值的权重和计算的，其中分配给每个值的权重是通过查询与相应键的兼容性函数计算的。

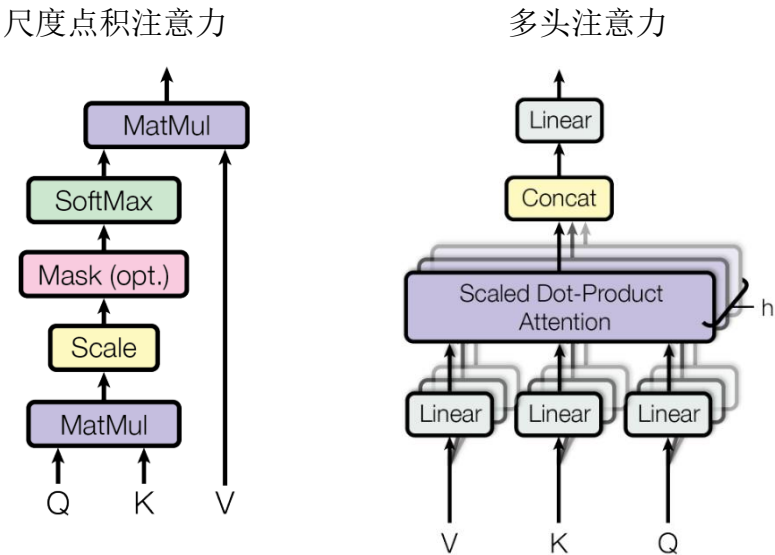


图 2: (左)尺度点积注意力。(右)多头注意力由几个并行运行的注意层组成。

3.2.1 尺度点积注意力

我们把我们特别的注意力称为“Scaled Dot-Product attention”（图 2）。

输入由查询、键的 d_k 维度、和值的 d_v 维度组成。我们计算这种查询与所有键的点积，再每个点积结果除以 $\sqrt{d_k}$ ，并应用 softmax 函数来获取的权重值。

在实践中，我们同时计算一组查询的注意函数，这些查询打包成一个矩阵 Q 。键和值也打包成矩阵 K 和 V 。我们计算输出的矩阵为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

最常用的两个注意函数是加性注意[2]和点积(乘法)注意。点积注意力与我们的算法完全相同，除了 $\frac{1}{\sqrt{d_k}}$ 的缩放因子。加性注意使用带有单个隐藏层的前馈网络来计算兼容性函数。虽然两者在理论上的复杂性相似，但点积注意力在实践中要快得多，而且空间效率更高，因为它可以使用高度优化的矩阵乘法代码来实现。

当 d_k 值较小时，两种机制表现相似，当 d_k 值较大时，不需缩放的加性注意优于点积注意[3]。我们猜想，当 d_k 值较大时，点积的增长幅度会变大，从而使 softmax 函数进入其梯度极小的区域。为了抵消这种影响，我们用 $\frac{1}{\sqrt{d_k}}$ 来缩放点积。

3.2.2 多头注意力

我们发现，将查询、键和值分别用不同的、学习过的线性投影 h 次地线性投影到 d_k 、 d_k 和 d_v 维上，比用 d_{model} 维度的键、值和查询执行单一的注意函数更为有效。然后，在查询、键和值的每一个版本上，我们并行执行注意力函数，产生 d_v 维的输出值。这些被连接起来并再次投影，最终得到图 2 所示的值。

⁴为了说明为什么点积会变大，假设 q 和 k 的分量是均值为 0，方差为 1 的独立随机变量。那么它们的点积， $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ ，其均值为 0，方差为 d_k 。

多头注意使得模型能够联合地注意不同位置上不同表示子空间的信息。如果只集中注意力，平均就会抑制这一点。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

其中，预测的是参数矩阵 $W_i^Q \in R^{d_{model} \times d_k}$, $W_i^K \in R^{d_{model} \times d_k}$ and $W_i^V \in R^{d_{model} \times d_v}$ 。

在这项工作中，我们采用了 $h = 8$ 个平行注意层或头。对于每一个，我们使用 $d_k = d_v = d_{model}/h = 64$ 。由于每个头的维度减小了，总的计算成本与全维度的单头注意相似。

3.2.3 在我们的模型中注意力的应用

Transformer 以三种不同的方式使用多头注意力：

- 在“encoder-decoder attention”层中，查询是来自之前的解码器层的，而记忆键和值来自编码器的输出。这使得解码器中的每个位置都能注意到输入序列中的所有位置。这模仿了序列到序列模型中典型的编码器-解码器注意机制，如[38, 2, 9]。
- 编码器包含自注意力层。在自注意力层中，所有的键、值和查询都来自同一个地方，在本例中，即编码器中前一层的输出。所述编码器中的每个位置可以注意到所述编码器的前一层中的所有位置。
- 类似地，解码器中的自注意力层允许解码器中的每个位置处理解码器中直到并包括该位置的所有位置。我们需要防止信息在解码器中向左流动，以保持自回归特性。我们通过屏蔽(设置为-1)softmax 输入中与非法连接对应的所有值来实现缩放点乘注意。 参见图 2。

3.3 Position-wise 前馈网络

除了注意子层外，我们的编码器和解码器中的每一层都包含一个全连接的前馈网络，该网络分别并相同地应用于每个位置。这包括两个线性转换，中间有一个 ReLU 激活函数。

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

虽然在不同的位置上线性转换是相同的，但它们在不同的层中使用不同的参数。这种方法的另一种说法是两个卷积，内核大小为 1。输入输出维数为 $d_{model} = 512$ ，内层维数为 $d_{ff} = 2048$ 。

3.4 Embeddings 层和 Softmax 激活函数

与其他序列转换模型类似，我们使用学习过的 Embedding 层将输入标记和输出标记转换为维度向量 d_{model} 。我们还使用常规学习过的线性变换层和 softmax 函数将解码器输出转换为预测的下一个标记的概率。在我们的模型中，我们共享两个 Embedding 层之间的权值矩阵和 pre-softmax 线性变换，这类似于[30]。在 Embedding 层中，我们将这些权重乘以 $\sqrt{d_{model}}$ 。

3.5 位置编码 (Positional Encoding)

由于我们的模型不包含递归和卷积，为了让模型利用序列的顺序，我们必须注入一些关于序列中记号的相对或绝对位置的信息。为此，我们将“位置编码”添加到编码器和解码器堆栈底部的 Embedding 层中。位置编码与 Embedding 层具有相同的维数模型，因此可以将两者相加。这里有许多位置编码的选择，基于学习和固定的[9]。

表 1:不同层类型的最大路径长度、每层复杂度和最小顺序操作次数。n 为序列长度，d 为表示维数，k 为卷积的核大小，r 为受限自注意的邻域大小。

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

在这项工作中，我们使用不同频率的正弦和余弦函数：

$$PE_{(pos;2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos;2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

其中 pos 是位置，i 是维度。也就是说，位置编码的每个维度都对应一个正弦信号。波长从 2π 到 $10000 \cdot 2\pi$ 呈几何级数。我们选择这个函数是因为我们假设它可以让模型很容易通过相对位置学习到注意力，因为对于任何固定偏移量 k， PE_{pos+k} 可以表示为一个线性函数 PE_{pos} 。

我们还使用学习过的 positional Embedding 层[9]进行了实验，发现两个版本产生了几乎相同的结果(见表 3 行(E))。我们选择正弦版本是因为它可以让模

型外推到比训练中遇到的序列更长的序列长度。

4 为什么使用 self-attention

在本节中，我们将自我注意层的各个方面与循环和卷积层进行比较，这些层通常用于映射一个可变长度的符号表示序列 $(x_1, \dots, x_n)$ 到另一个等长序列 $(z_1, \dots, z_n)$ ， x_i 和 z_i 都属于 $\mathbf{R}^d$ ，例如一个隐藏层在一个典型的序列转导编解码器中。激励我们使用 self-attention，我们考虑三个需求。

一是每一层的总计算复杂度。另一个是可以并行化的计算量，由所需的最小顺序操作次数来衡量。第三点是网络中远程依赖关系之间的路径长度。在许多序列转换任务中，学习长期依赖是一个关键的挑战。影响学习这种依赖关系的能力的一个关键因素是信号在网络中必须通过的前向和后向路径的长度。输入和输出序列中任意位置组合之间的路径越短，学习远程依赖关系[12]就越容易。因此，我们也比较了由不同层类型组成的网络中任意两个输入和输出位置之间的最大路径长度。

如表 1 所示，self-attention 层用固定数量的顺序执行操作连接所有位置，而循环层需要 $O(n)$ 个顺序操作。在计算复杂性方面，当序列长度 n 小于表示维数 d 时 self-attention 层速度比循环层快，这是机器翻译中最先进模型的句子表示的最常见的例子，如 word-piece[38]和 byte-pair[31]表示。为了提高非常长的序列的任务的计算性能，可以将 self-attention 限制在只考虑输入序列中大小为 r 的以各自输出位置为中心的邻域。这将使最大路径长度增加到 $O(n/r)$ 。我们计划在未来的工作中进一步研究这种方法。

核的宽 $k < n$ 的单卷积层不连接所有的输入输出位置。这样做，在相邻核的情况下需要一个 $O(n/k)$ 卷积层的堆栈，或在空洞卷积[18]的情况下需要 $O(\log_k(n))$ ，并会增加网络中任意两个位置之间的最长路径的长度。卷积层通常比循环层代价更高，这按照 k 的系数计算。然而，可分离卷积[6]大大降低了复杂度，达到 $O(k \cdot n \cdot d + n \cdot d^2)$ 。然而，即使在 $k = n$ 的情况下，可分离卷积的复杂度也等价于 self-attention 层和点向前馈层 (point-wise feed-forward layer) 的结合，这是我们在模型中采用的方法。

作为附带的好处，self-attention 可以产生更多可解释的模型。我们检查了我们模型中的注意力分布，并在附录中展示和讨论了一些例子。不仅个别注意头

清楚地学会了执行不同的任务，许多注意头似乎表现出与句子的句法和语义结构有关的行为。

5 训练

本节描述了我们模型的训练方法。

5.1 训练数据和批处理

我们使用由 450 万对句子组成的 WMT 2014 英德标准数据集进行训练。使用 [3] 编码的字节对对句子进行编码，该编码具有大约 37000 个标记的共享源目标词汇表。对于英语-法语，我们使用的是明显更大的 WMT 2014 年英法数据集，包含 3600 万个句子，并将标记分解成 32000 个单词块词汇[38]。句子对按照近似的序列长度进行分组。每个训练 batch(批)包含一组句子对，其中包含大约 25000 个源标记和 25000 个目标标记。

5.2 硬件和进度

我们在一台装有 8 个 NVIDIA P100 gpu 的机器上训练我们的模型。对于使用本文中描述的超参数的基本模型，每个训练步骤大约需要 0.4 秒。我们对基本模型进行了总共 10 万步（12 小时）的训练。对于我们的大型模型(如表 3 所示)，每步时长为 1.0 秒。这些大模型被训练 30 万步(3.5 天)。

5.3 优化器

我们使用了 Adam 优化器[20]， $\beta_1 = 0.9$ ， $\beta_2 = 0.98$ 和 $\epsilon = 10^{-9}$ 。我们根据以下公式改变了学习率：

$$lrate = d_{model}^{-0.5} \cdot \min(step_{num}^{-0.5}, step_{num} \cdot warmup_{steps}^{-0.5}) \quad (3)$$

这对应于第一个 warmup_steps 训练步骤的学习速率线性增加，然后与步数的平方根的倒数成比例减少。我们使用 warmup_steps = 4000。

5.4 正则化

在训练过程中，我们采用了三种正则化方法：

残差丢弃

我们对每个子层的输出应用 dropout[33]，然后将其添加到子层输入并进行规范化。此外，我们在编码器和解码器堆栈中对嵌入和位置编码的和应用了 dropout。对于基础模型，我们使用的速率为 $P_{drop} = 0.1$ 。

表 2: transformer 在英语-德语和英语-法国 newstest2014 一小部分训练代价测试中比之前的最先进的型号更,。

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

标签平滑

在训练过程中，我们使用了 $\epsilon_{ls} = 0.1$ [36] 值的标签平滑。这会影响 perplexity，因为模型学习了更多的不确定性，但提高了准确性和 BLEU 得分。

6 结果

6.1 机器翻译

在 WMT 2014 英->德的翻译任务中，大的 transformer 模型(表 2 中的 Transformer (big))比之前已公开的最好的模型(包括全部)的表现超过 2.0 BLEU，创造了新的最先进的 BLEU 评分 28.4。该模型的配置列于表 3 的底部。在 8 个 P100 GPU 上的训练时间为 3.5 天。甚至我们的基础模型超过了所有以前发表的模型和集合，而训练成本只是任何竞争模型的一小部分。

在 WMT 2014 英->法翻译任务中，我们的大模型的 BLEU 得分为 41.0，超过了之前所有发表过的单个模型，而且训练成本还不到之前最先进的模型的 1/4。Transformer(big)模型的英->法训练使用的丢弃率 $P_{drop} = 0.1$ ，而不是 0.3。

对于基础模型，我们使用了通过获取最后 5 个检查点 (checkpoints) 平均值的单个模型，这些检查点每 10 分钟编写一次。对于大型模型，我们计算了最后

20 个检查点的平均值。我们使用了波束 (beam) 大小为 4, 惩罚长度 $\alpha = 0.6$ [38] 的定向搜索 (beam search)。这些超参数是在开发台上经过实验后选定的。在推理期间, 我们将最大输出长度设置为输入长度+ 50, 但在可能的情况下提前终止 [38]。

表 2 总结了我们的结果, 并将我们的翻译质量和培训成本与文献中的其他模型架构进行了比较。我们通过将训练时间、所使用的 GPU 数量和每个 GPU 5 的持续单精度浮点容量相乘, 来估计用于训练一个模型的浮点运算的数量。

6.2 模型变体

为了评估 Transformer 不同组件的重要性, 我们以不同的方式改变了我们的基础模型, 并在开发集《newstest2013》上测试了英德翻译的性能变化。我们使用了前一节所描述的定向搜索, 但没有使用检查点平均值。我们在表 3 中展示了这些结果。

在表 3 中的行 (A) 中, 我们改变注意头的数量以及注意键和维度的值, 保持计算量不变, 如章节 3.2.2 所述。虽然单头注意力比最好的设置差 0.9 BLEU, 但过多的注意力头也会降低质量。

⁵我们分别使用 2.8、3.7、6.0 和 9.5 作为 K80、K40、M40 和 P100 的 TFLOPS。

表 3: Transformer 架构的变化。未列出的值与基础模型相同。所有指标都在英德翻译开发集 newstest2013 上测试的。根据我们的字节对编码, perplexities 是逐字片 (per-wordpiece) 的, 不应该与逐字 (per-word) 的 perplexities 相比较。

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)					1	512				5.29	24.9	
					4	128				5.00	25.5	
					16	32				4.91	25.8	
					32	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
	256				32	32				5.75	24.5	28
	1024				128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
								0.0		5.77	24.6	
(D)								0.2		4.95	25.5	
									0.0	4.67	25.3	
									0.2	5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16				0.3	300K	4.33	26.4	213

表4:Transformer很好地概括了英语选区解析(English constituency parsing)
(结果见 WSJ 第 23 节)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

在表 3 的行(B)中，我们观察到降低注意键大小 d_k 会影响模型质量。这表明确定兼容性并不容易，一个比点积更复杂的兼容性函数可能是有益的。我们在(C)和(D)行进一步观察到，正如预期的那样，更大的模型更好，并且 dropout 在避免过度拟合方面非常有帮助。在行(E)中，我们将正弦位置编码替换为学习过的位置 embeddings [9]，并观察到与基模型几乎相同的结果。

6.3 English Constituency Parsing 英语选区解析

为了评估 Transformer 是否可以推广到其他任务，我们对英语分组解析进行了实验。这项任务提出了具体的挑战：输出受到强烈的结构性限制，而且明显长

于输入。此外，RNN 序列到序列模型还不能在小数据区[37]中获得最先进的结果。

我们在《华尔街日报》(WSJ)中的宾夕法尼亚树银行部分训练了一个 $d_{model} = 1024$ 的 4 层 transformer [25]，大约有 40K 个训练句子。我们还在半监督环境中用包含大约 17M 个句子的更大的高置信度和 BerkleyParser 语料库[37] 训练它。我们在《华尔街日报》的设置中使用了 16K 个标记词汇表，在半监督设置中使用了 32K 个标记词汇表。

我们都只进行了少量的实验，选择注意和残差中的丢弃(dropout) (第 5.4 节)、第 22 节的学习率和波束尺寸、所有其他参数从英语到德语的基础翻译模型保持不变。在推理过程中，我们将最大输出长度增加到输入长度+ 300。我们使用 21 和 $\alpha = 0.3$ 的光束大小的华尔街日报和半监督设置。

表 4 中的结果显示，尽管缺少特定于任务的调优，但我们的模型的性能令人惊讶地好，产生的结果比之前公布的所有模型更好，除外 Recurrent Neural Network Grammar [8]。

与的 RNN 序列到序列模型[37]相比，Transformer 甚至在仅使用《华尔街日报》40K 句的训练集进行训练时，也比 BerkeleyParser[29]表现更好。

7 结论

在本工作中，我们提出了 Transformer，这是第一个完全基于注意力的序列转换模型，用多头自注意替换了编码器-解码器架构中最常用的循环层。

对于翻译任务，Transformer 的训练速度比基于循环或卷积层的体系结构快得多。在 2014 年世界 WMT 英德和 2014 年世界 WMT 上英法翻译任务，我们达到了新的艺术境界。在前一个任务中，我们的最佳模型甚至优于所有之前公布的全部。

我们对基于注意力的模型的未来感到兴奋，并计划将其应用到其他任务中。我们计划将 Transformer 扩展到涉及文本以外的输入和输出模式的问题，并研究局部受限的注意力机制，以有效地处理图像、音频和视频等大型输入和输出。减少迭代顺序是我们的另一个研究目标。我们用来训练和评估模型的代码可以在 <https://github.com/tensorflow/tensor2tensor> 网站获得

感谢

我们感谢 Nal Kalchbrenner 和 Stephan Gouws 丰富的评论、更正和启发。

参考文献

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.
- [5] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.
- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.
- [10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*,

9(8):1735–1780, 1997.

[14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.

[15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.

[16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.

[17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.

[18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.

[19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.

[20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.

[22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.

[23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.

[24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attentionbased neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.

[25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.

[26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages

152–159. ACL, June 2006.

[27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.

[28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.

[29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.

[30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.

[31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.

[32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.

[35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.

[36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.

[37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.

[38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation

system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

[39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.

[40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

注意力可视化

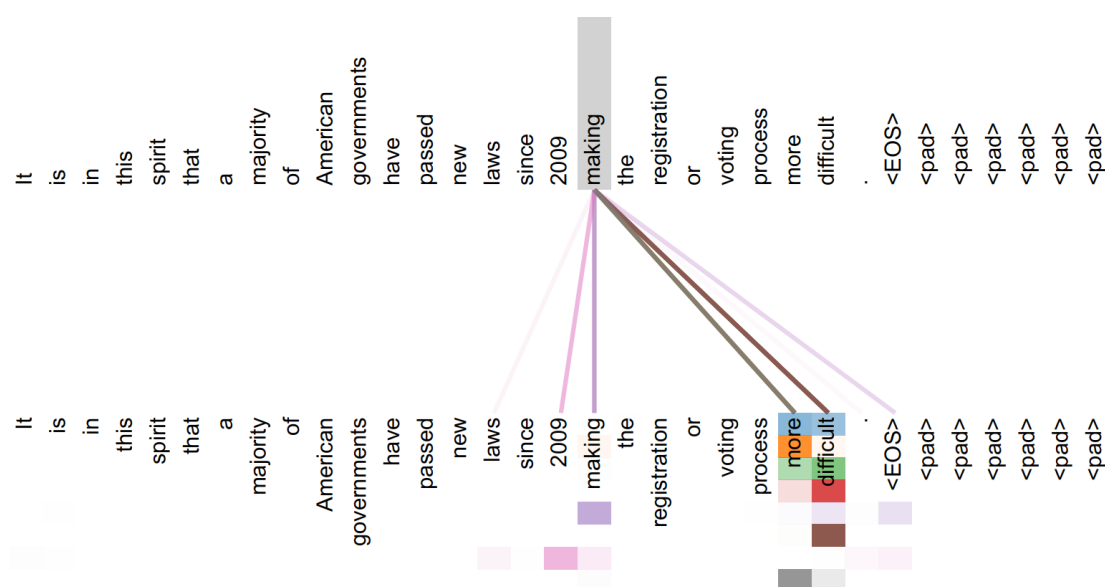


图 3:在第 6 层的第 5 层中，编码器的远距离依赖的注意力机制的一个例子。许多注意力头部集中在动词 making 的远距离依赖关系上，从而完成词组 “making ... more difficult”。这里只展示了 “making” 这个词。不同的颜色代表不同的头部。最好的视觉效果用颜色标出。

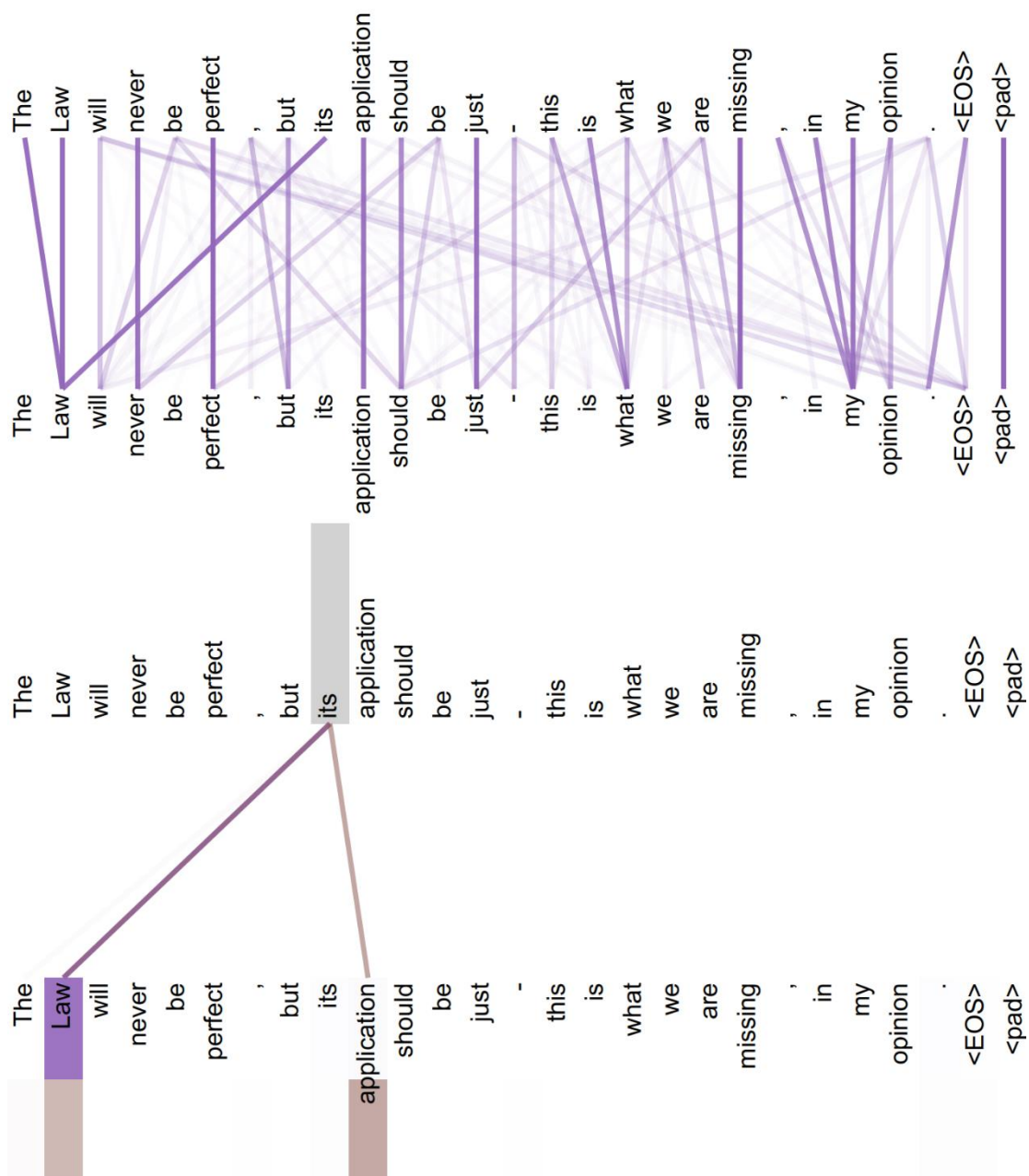


图 4：也在第 5 层和第 6 层的两个注意力头部，显然涉及回指解决。上图:注意 5 号头。底部:将注意力从“its”这个单词中分离出来作为第 5 和第 6 层的注意力头部。注意，对这个词的关注非常强烈。

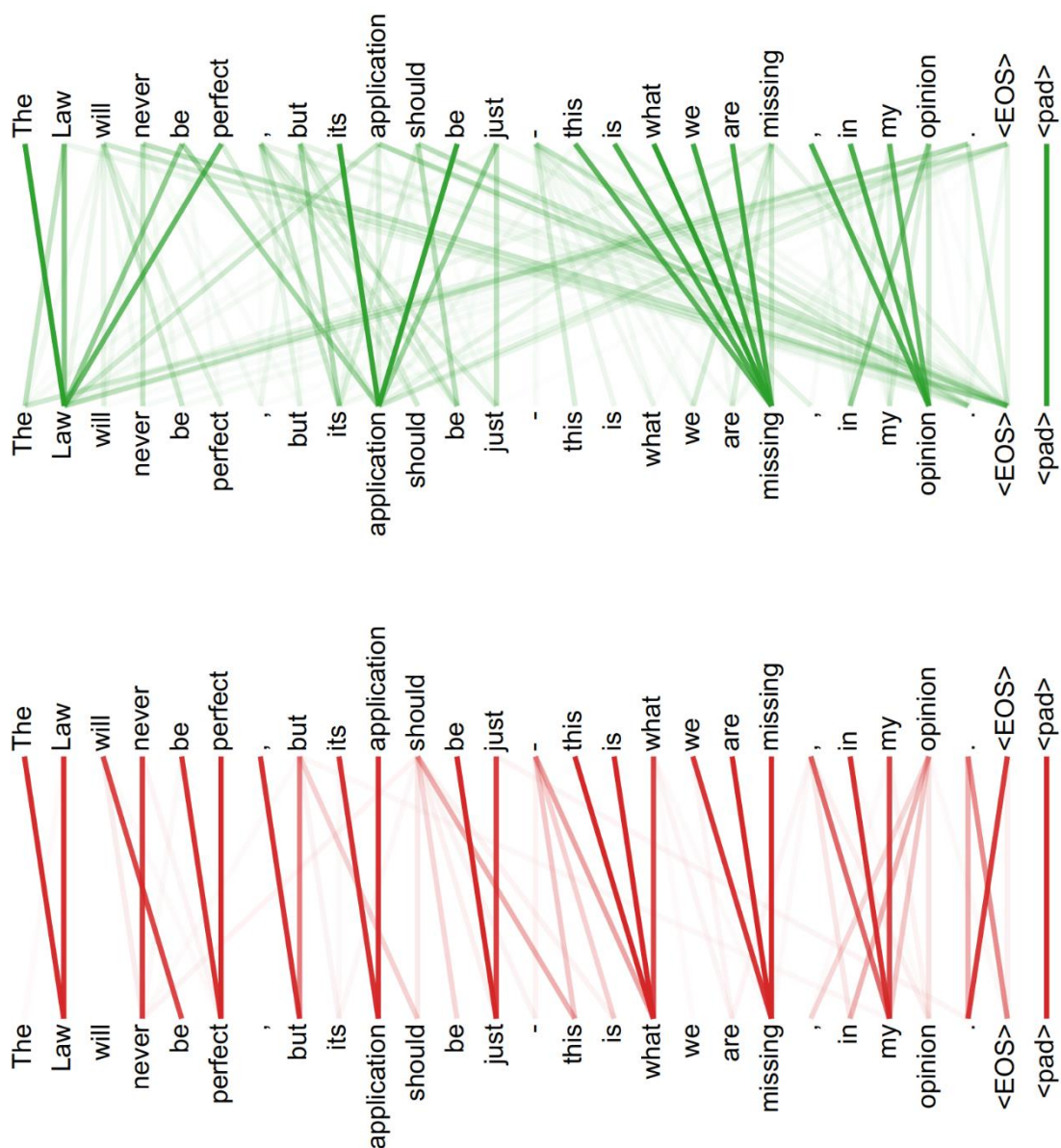


图 5:许多注意力头部表现出的行为似乎与句子的结构有关。我们在上面给出了两个这样来自 6 层的编码器自注意力的两个不同的头部的例子。这些头部显然学会了执行不同的任务。